

판다스(Pandas) DataFrame

값/영역 접근 & 추가/삭제 & 연산 강의 교재

광명코딩학원 | 파이썬 판다스 기초 강의

[핵심 개념] DataFrame에서 특정 값/영역 접근, 컬럼/로우 추가 · 삭제, 브로드캐스팅 · 필터링 · 컬럼 시프트 연산 방법을 배웁니다.

PART 1. 특정 값 / 영역 가져오기

1. 특정 값 가져오기

[개념] DataFrame에서 특정 위치의 단일 값을 가져올 때 `iloc`[행번호, 열번호] 또는 `loc`[인덱스, 컬럼명]을 사용합니다. 행 먼저, 열 나중!

특정 값 접근

```
from pandas import DataFrame

data = [['3R',1510,7.36], ['3SOFT',1790,1.65], ['ACTS',1185,1.28]]

index = ['037730','036360','005760']

columns = ['종목명','현재가','등락률']

df = DataFrame(data=data, index=index, columns=columns)

# iloc - - 행번호, 열번호로 접근

print(df.iloc[1, 1]) # 1790 (행1, 열1)

# loc - - 인덱스, 컬럼명으로 접근

print(df.loc['036360', '현재가']) # 1790
```

방법	문법	기준
iloc	df.iloc[행번호, 열번호]	숫자 위치 (행번호, 열번호)
loc	df.loc['인덱스', '컬럼명']	레이블 이름

2. 특정 영역 가져오기

영역 접근

```
# iloc - - 리스트로 선택

print(df.iloc[[0, 1], [0, 1]])

# 종목명 현재가

# 037730 3R 1510

# 036360 3SOFT 1790

# iloc - - 슬라이싱 (끝 미포함, 동일 결과)

print(df.iloc[0:2, 0:2])

# loc - - 리스트로 선택

print(df.loc[['037730','036360'], ['종목명','현재가']])

# loc - - 슬라이싱 (끝 포함)

print(df.loc['037730':'036360', '종목명':'현재가'])
```

비연속 영역

```
# 비연속 영역 (슬라이싱 불가 - > 리스트 필수)

# 종목명(0번)과 등락률(2번) - - 붙어있지 않음

print(df.iloc[[0, 1], [0, 2]])

print(df.loc[['037730','036360'], ['종목명','등락률']])

# 종목명 등락률

# 037730 3R 7.36
```

용도	iloc	loc
특정 값	df.iloc[1, 1]	df.loc['036360','현재가']
영역(리스트)	df.iloc[[0,1],[0,1]]	df.loc[['037730','036360'],['종목명','현재가']]
영역(슬라이싱)	df.iloc[0:2,0:2] 끝 미포함	df.loc['A':'B','C':'D'] 끝 포함

PART 2. 컬럼 / 로우 추가 & 삭제

3. 컬럼 추가

[개념] `df['새컬럼명'] = 시리즈 또는 연산값` 으로 새 컬럼을 추가합니다. 브로드캐스팅으로 모든 행에 한꺼번에 적용됩니다.

컬럼 추가 예제

```
import pyupbit

df = pyupbit.get_ohlcv('KRW - BTC') # 비트코인 일봉 데이터

# range 컬럼 추가 (고가 - 저가)

df['range'] = df['high'] - df['low']

# 결과 (상위 3행)

# open high low close range

# 2020 - 09 - 17 09:00:00 12908000 13050000 12623000 12811000 427000.0

# 2020 - 09 - 18 09:00:00 12811000 12837000 12672000 12777000 165000.0

# 2020 - 09 - 19 09:00:00 12781000 13036000 12741000 12940000 295000.0
```

- `df['high'] - df['low']` - > 같은 인덱스끼리 빼줌 (브로드캐스팅)
- 반복문 없이 전체 행에 한꺼번에 적용
- 이미 있는 컬럼명이면 수정, 없는 컬럼명이면 추가

4. 컬럼 삭제 (drop axis=1)

[개념] `df.drop('컬럼명', axis=1)` 로 컬럼을 삭제합니다. `axis=1`은 컬럼 방향을 의미합니다. 기본적으로 원본은 유지되며 `inplace=True` 시 원본 변경.

컬럼 삭제 예제

```
# 단일 컬럼 삭제 (원본 유지, 새 DataFrame 리턴)
```

```
df2 = df.drop('volume', axis=1)

# 여러 컬럼 동시 삭제

df2 = df.drop(['volume', 'range'], axis=1)

# inplace=True: 원본 직접 변경

df.drop('volume', axis=1, inplace=True)

# df에서 volume 컬럼이 바로 삭제됨
```

파라미터	값	의미
axis=0	행 방향	로우(행) 삭제
axis=1	컬럼 방향	컬럼(열) 삭제
inplace=True	-	원본 직접 변경 (없으면 새 DF 리턴)

5. 시계열 데이터와 인덱스 (to_datetime)

[개념] 날짜/시간 데이터를 인덱스로 사용할 때는 문자열이 아닌 DatetimeIndex 또는 Timestamp 타입으로 변환해야 합니다.

to_datetime 변환

```
import pandas as pd

# 문자열 리스트 -> DatetimeIndex

date = ['2021-04-09']

index = pd.to_datetime(date)

print(type(date)) #

print(type(index)) #

# 단일 문자열 -> Timestamp

date = '2021-04-09'
```

```
index = pd.to_datetime(date)

print(type(index)) #
```

입력	결과 타입	용도
단일 문자열 '2021 - 04 - 09'	Timestamp	로우 추가 시 인덱스
문자열 리스트 ['2021 - 04 - 09']	DatetimeIndex	여러 날짜 인덱스

6. 로우 추가 (loc 사용)

로우 추가 예제

```
import pandas as pd

# 새 날짜 인덱스 생성 (Timestamp 타입으로 변환)

date = '2022 - 01 - 01'

index = pd.to_datetime(date)

# 로우 추가 - - df.loc[인덱스] = 데이터

df.loc[index] = [100, 100, 100, 100, 100]

# 결과: 마지막 행에 2022 - 01 - 01 데이터 추가됨
```

- loc[인덱스] = [값, 값, ...] 형태로 추가
- 인덱스는 반드시 to_datetime()으로 변환한 Timestamp 타입 사용
- 해당 인덱스가 없으면 추가, 있으면 수정

7. 로우 삭제 (drop axis=0)

로우 삭제 예제

```
import pyupbit

df = pyupbit.get_ohlcv('KRW - BTC')

# 마지막 로우 인덱스 가져오기
```

```
index = df.index[ - 1] # 거래일 당일의 인덱스
```

```
# 당일 데이터 로우 삭제 (원본 유지)
```

```
df.drop(index, axis=0)
```

```
# 원본에 바로 반영
```

```
df.drop(index, axis=0, inplace=True)
```

- axis=0 → 행(로우) 방향 삭제
- df.index[- 1] 로 마지막 인덱스 가져오기
- inplace=True 없으면 원본은 그대로 유지

PART 3. DataFrame 기초 연산

8. 브로드캐스팅 (Broadcasting)

[개념] DataFrame에서 특정 컬럼을 선택하면 Series가 됩니다. 이 Series에 연산을 적용하면 모든 값에 동시 적용됩니다.

브로드캐스팅

```
import pyupbit

df = pyupbit.get_ohlcv('KRW - BTC')

# open 컬럼(Series)의 모든 값에 100 더하기

print(df['open'] + 100)

# 2021-03-31 70800100.0

# 2021-04-01 71480100.0

# 2021-04-02 72749100.0 ...

# 컬럼끼리 연산 (같은 인덱스끼리 자동 매핑)

df['range'] = df['high'] - df['low'] # 컬럼 추가와 동시에 연산
```

- `df['컬럼'] + 100` → 해당 컬럼의 모든 값에 +100
- `df['high'] - df['low']` → 같은 날짜(인덱스)끼리 자동으로 빼줌
- 반복문 없이 전체 데이터에 일괄 연산 가능

9. DataFrame 필터링

[개념] 비교 연산자의 결과는 Boolean(True/False) 값을 가진 Series입니다. 이를 DataFrame 인덱싱에 넣으면 True인 로우만 필터링됩니다.

DataFrame 필터링

```
import pyupbit
```



```

df = pyupbit.get_ohlcv('KRW - BTC')

# 종가가 시가보다 높은 날 (상승한 날) 필터링

cond = df['close'] > df['open'] # Boolean Series 생성

print(cond)

# 2020 - 09 - 17 False

# 2020 - 09 - 18 False

# 2020 - 09 - 19 True ...

# cond로 True인 로우만 필터링

result = df[cond]

print(result.shape) # (99, 6) - - 200개 중 99일이 상승

# 한 줄로 줄이기

result = df[df['close'] > df['open']]

```

- 조건 생성 → Boolean Series 리턴
- df[cond] → True인 행만 남긴 새 DataFrame 리턴
- 200일 중 약 49.5%인 99일이 상승 (약 50:50)

10. 컬럼 시프트 (shift)

[개념] DataFrame의 연산은 기본적으로 같은 인덱스의 컬럼과 컬럼 사이에 적용됩니다. 당일 값 - 전일 값처럼 위아래로 빼려면 shift()를 사용합니다.

shift 문제 상황 — 위아래로 빼기

shift 필요성

```

# 문제: 당일 저가 - 전일 저가를 구하려면?

# df['low'] - df['low'] < - 이렇게는 같은 날끼리만 빼짐

# 위아래(상하)로 빼는 방법이 필요!

```

shift() 사용법

shift 예제

```
# df['컬럼명'].shift(n) - - 컬럼 데이터를 n칸 아래로 이동

# close_shift1 컬럼 생성 (close를 1칸 아래로 이동)

df['close_shift1'] = df['close'].shift(1)

# 결과

# close close_shift1

# 2020-09-17 09:00:00 12811000 NaN < - 올라올 값 없음

# 2020-09-18 09:00:00 12777000 12811000.0 < - 전날 close 값

# 2020-09-19 09:00:00 12940000 12777000.0

# 2020-09-20 09:00:00 12819000 12940000.0
```

shift 활용 — 전일 대비 증가 변화 계산

shift 활용 — 전일 대비 계산

```
# 증가 변화 = 오늘 증가 - 어제 증가

df['증가변화'] = df['close'] - df['close'].shift(1)

# 결과

# close close_shift1 증가변화

# 2020-09-17 09:00:00 12811000 NaN NaN

# 2020-09-18 09:00:00 12777000 12811000.0 -9.0 < - 오늘 - 어제

# 2020-09-19 09:00:00 12940000 12777000.0 35.0
```

shift 값	방향	의미
shift(1)	아래로 1칸	각 행에 전일(이전) 값이 들어옴
shift(2)	아래로 2칸	각 행에 2일 전 값이 들어옴

shift 값	방향	의미
shift(-1)	위로 1칸	각 행에 다음날 값이 들어옴

11. 주요 개념 정리

기능	문법/설명
특정 값 접근	df.iloc[행, 열] / df.loc[인덱스, 컬럼]
영역 접근	df.iloc[[0,1],[0,1]] / df.loc[['A','B'],['C','D']]
비연속 영역	슬라이싱 불가 - > 리스트 형태로 직접 지정
컬럼 추가	df['새컬럼'] = 값/연산 (브로드캐스팅 적용)
컬럼 삭제	df.drop('컬럼', axis=1) / axis=1=컬럼방향
시계열 인덱스	pd.to_datetime('날짜') - > Timestamp 변환 후 사용
로우 추가	df.loc[to_datetime인덱스] = [값...]
로우 삭제	df.drop(인덱스, axis=0) / axis=0=행방향
inplace	True시 원본 직접 변경 / False(기본)시 새 DF 리턴
브로드캐스팅	df['컬럼'] + 100 - > 모든 값에 동시 적용
필터링	cond=df['close']>df['open'] / df[cond] - > True행만
shift(n)	컬럼을 n칸 아래로 이동 - > 전일값 연산에 활용

[실습] 실습 팁: pyupbit 모듈 설치 후 실제 코인 데이터로 연습해 보세요. pip install pyupbit / df = pyupbit.get_ohlcv('KRW - BTC')